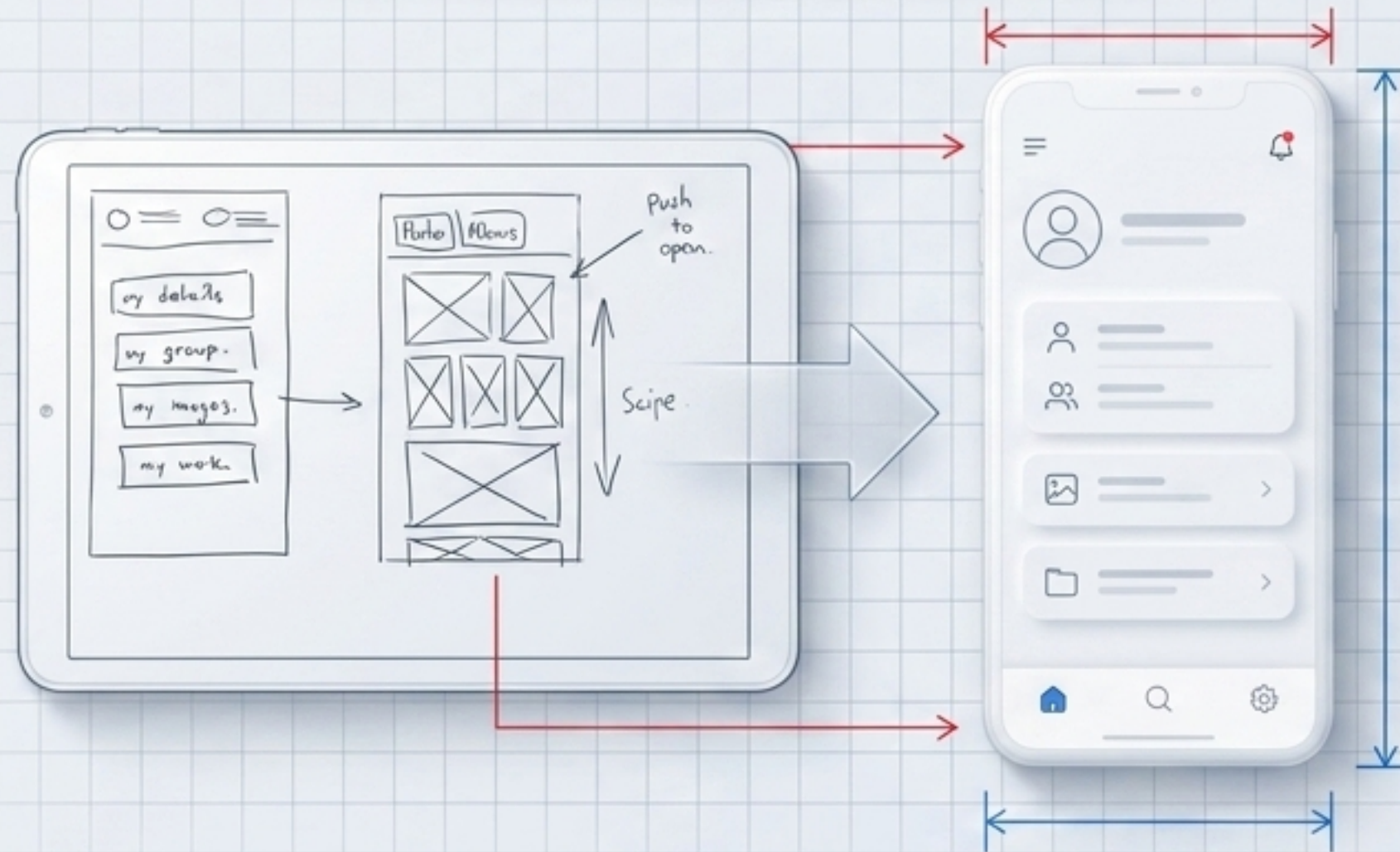




Interfaces Gráficas de Usuario en Java

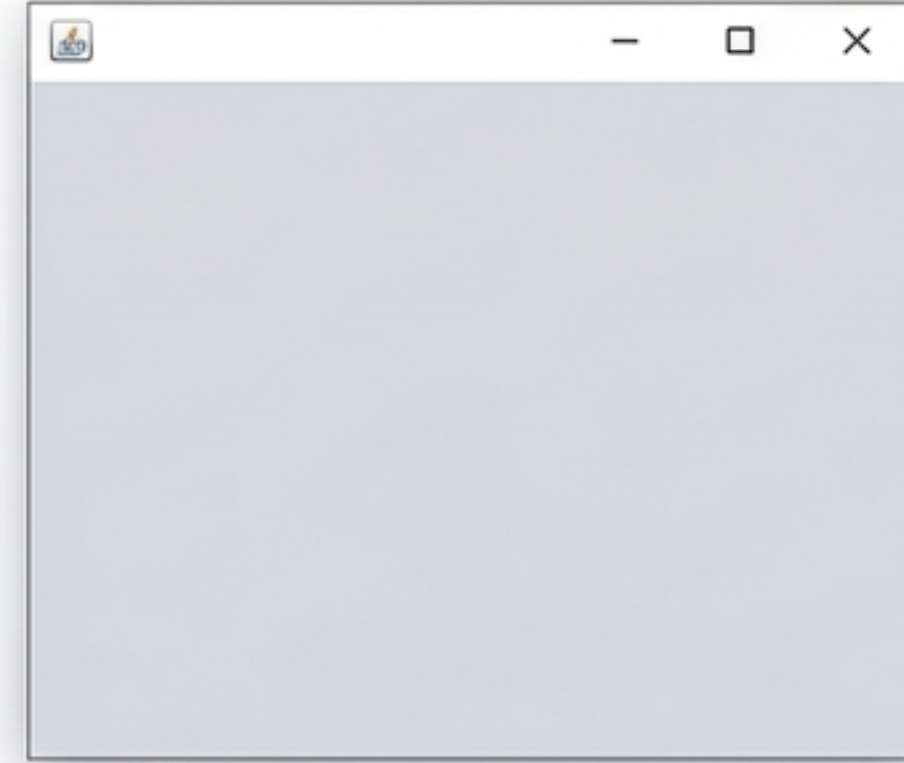
Construyendo aplicaciones visuales interactivas con la biblioteca SWING.

El Pasado

```
C:\> PROGRAMA.EXE  
INPUT DATE: 2024-05-15  
PROCESSING...  
RESULT: 42  
C:\>_
```

Entrada por teclado, ejecución lineal.

El Salto **Cualitativo**



Ecosistema visual e interactivo.

Preparación del Entorno IDE

1. Nuevo Proyecto
-> Java Application

2. Desmarcar
"Create main class"

3. Clic derecho en
paquete -> Nuevo
-> JFrame form

4. Nombrar ventana
(Hereda de JFrame)

5. Ejecutar y
establecer como
principal

Anatomía de la Interfaz

JLabel (Etiqueta).
Uso: Texto estático.

Usuario:

Passoneño:

Comentarios (opcional):

Login Cancel

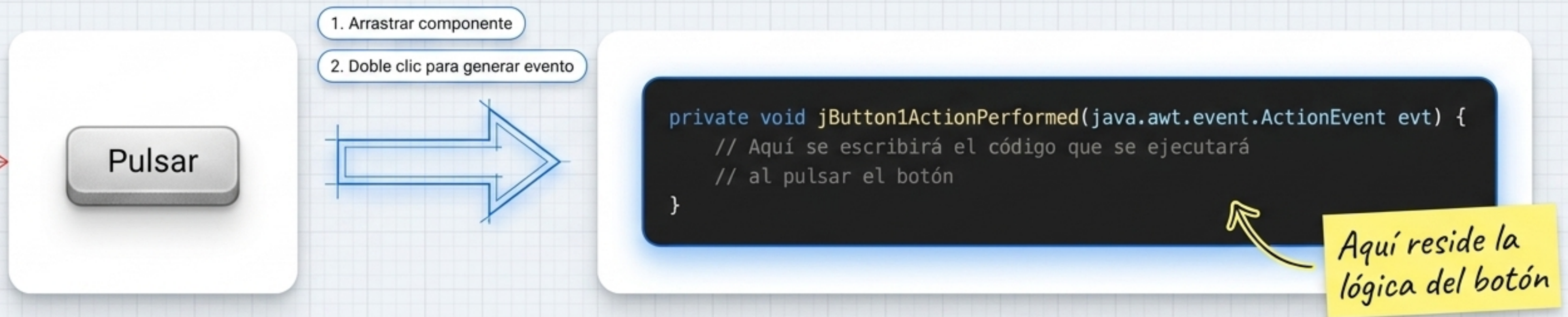
JTextField (Caja de texto).
Uso: Entrada de una sola línea.

JTextArea (Área de texto).
Uso: Entrada de múltiples líneas.

```
// Leer datos  
String texto = elemento.getText();  
  
// Escribir datos  
elemento.setText("Nuevo valor");
```


El Motor de la Acción: JButton

Los botones detonan acciones.
En SWING, la interactividad nace al
sobrecargar eventos.



Matriz de Selección de Usuario

JCheckBox



Tipo:
Selección múltiple
(ej. preferencias).

```
isSelected()
```

JRadioButton

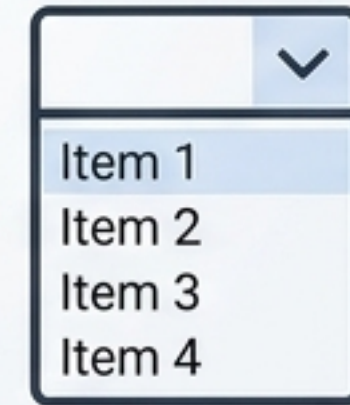
Grupo 1 Grupo 2

☐ Opción 1	☒ Opción 1
☐ Opción 2	☐ Opción 2
☒ Opción 3	☐ Opción 3

Tipo:
Selección única
excluyente.

```
Requiere: ButtonGroup
```

JComboBox



Tipo:
Lista de valores
empaquetada.

```
addItem() / getSelectedItem()
```


Estructuración Compleja: JTable

Para manipular la tabla, primero extraemos su "alma" (el modelo):

```
DefaultTableModel modelo = (DefaultTableModel) jtable.getModel();
```

Nombre	Edad	Género
Javi	25	M
Lucia	30	F
Fran	35	M



DefaultTableModel
(Datos y Lógica)

1. Insertar fila: `addRow(arrayStrings)`

2. Añadir columna: `addColumn("Nombre")`

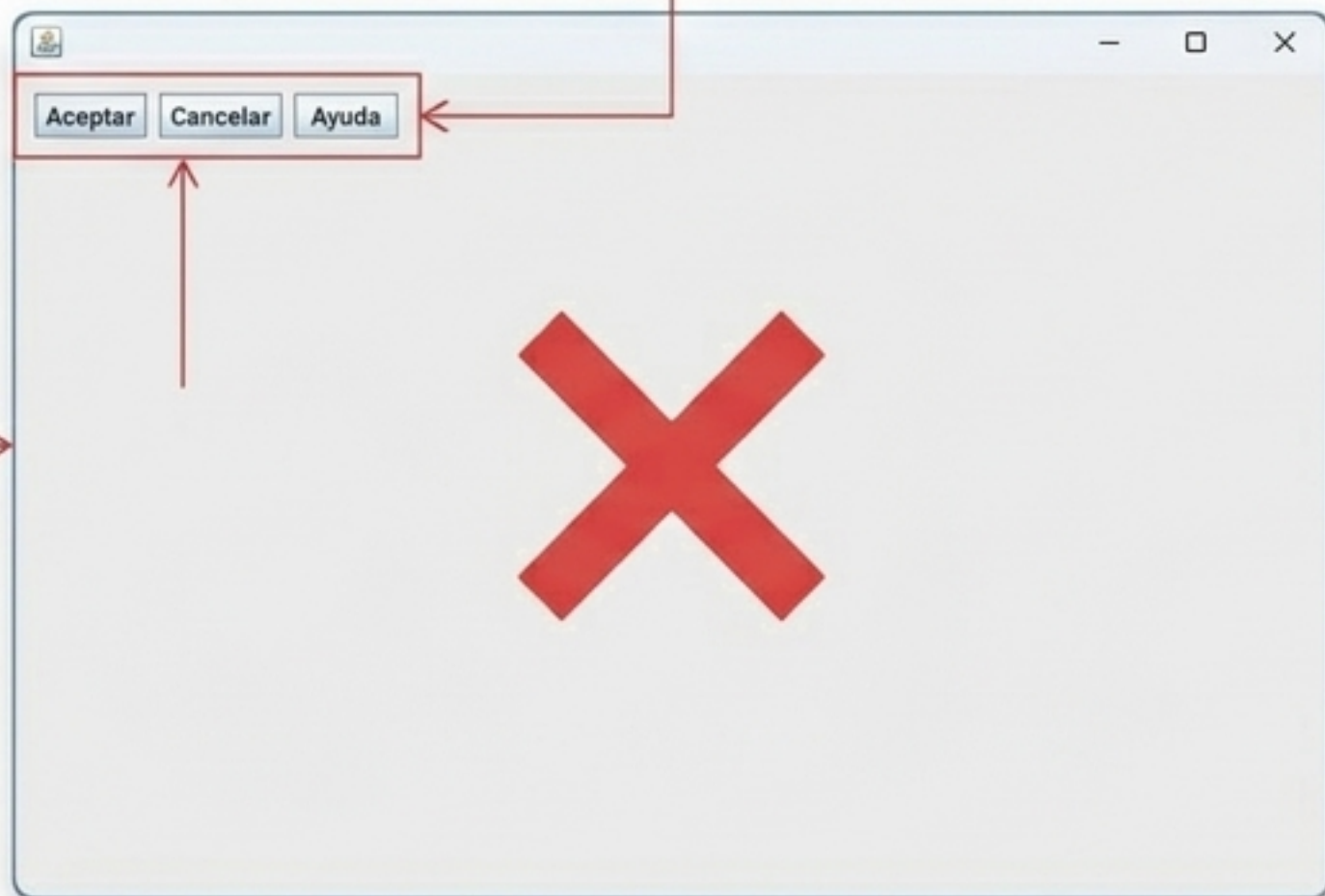
3. Eliminar fila: `removeRow(indice)`

4. Leer fila: `getSelectedRow()`

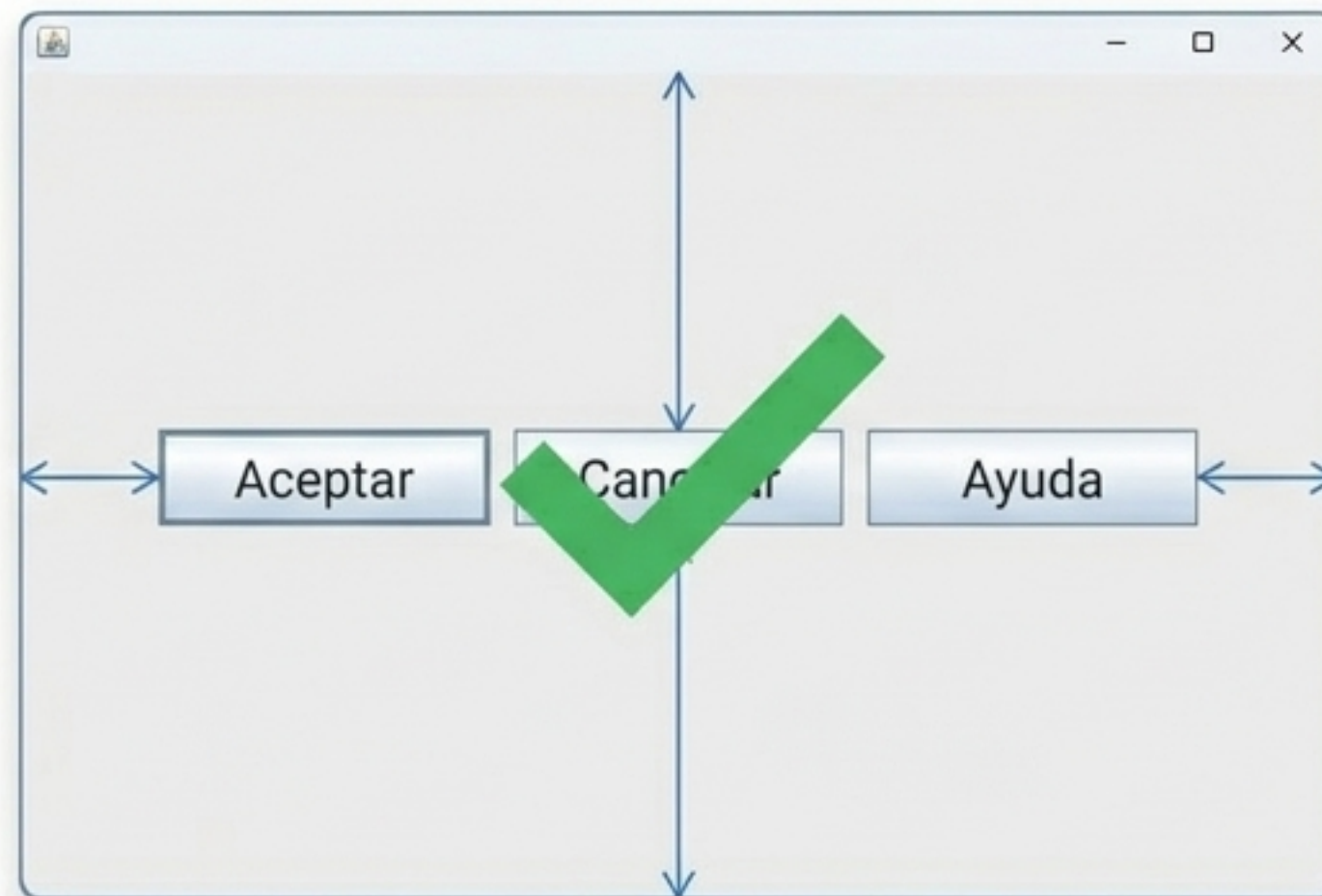
5. Leer columna: `getSelectedColumn()`

El Problema de la Adaptabilidad

El Error Común (Coordenadas Estáticas)



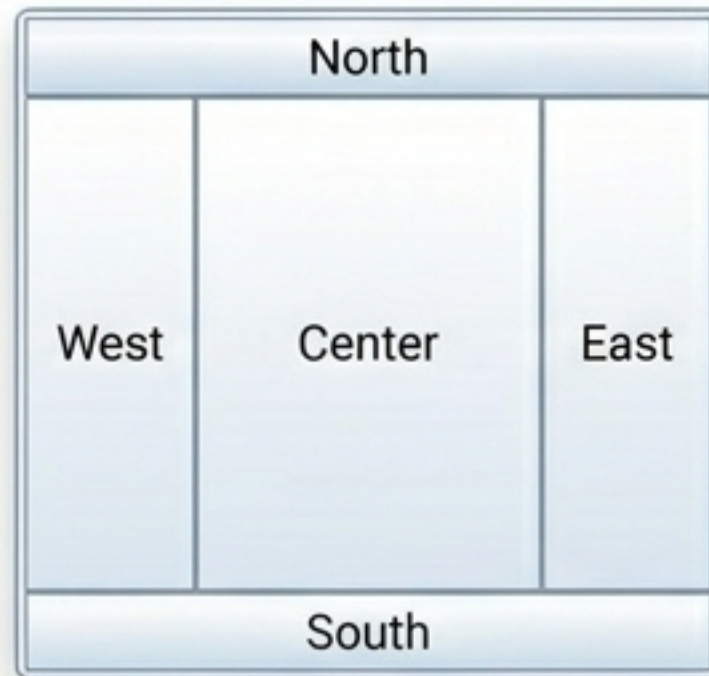
El Comportamiento Deseado (Layouts)



¿Por qué ocurre esto? Arrastrar y soltar fija coordenadas estáticas. Cuando el usuario cambia el tamaño de la ventana, la interfaz se rompe. La solución es automatizar la distribución mediante Layouts.

Diagnóstico de Layouts Fundamentales

BorderLayout



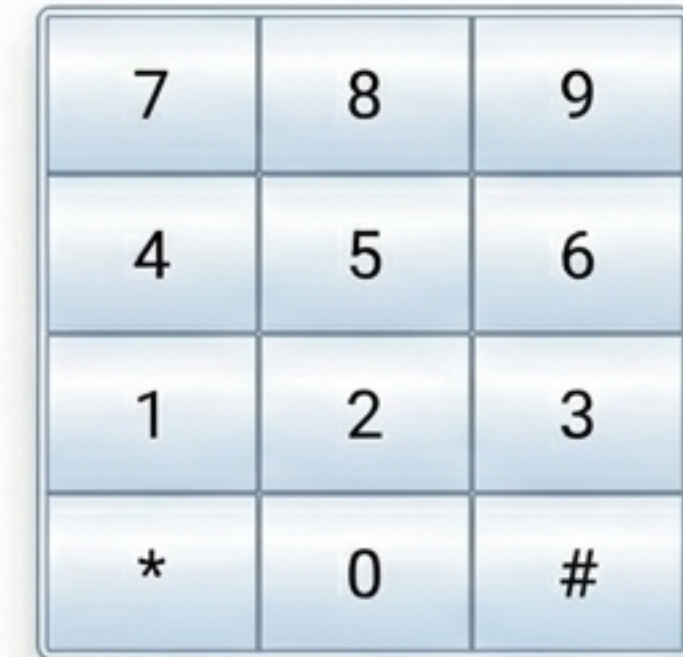
Uso: Interfaces clásicas con barras de herramientas y áreas principales. Permite anidamiento de JPanel.

FlowLayout



Uso: Distribución centrada y adaptable por defecto.

GridLayout



Uso: Estructuras rígidas proporcionales (Ej: teclados de calculadora).

Taxonomía de Mensajes del Sistema (Ventanas Modales)

Las ventanas modales bloquean el programa hasta que el usuario responde.
Clase base: `JOptionPane.showMessageDialog()`



`JOptionPane.INFORMATION_MESSAGE`



`JOptionPane.WARNING_MESSAGE`



`JOptionPane.ERROR_MESSAGE`

(Ideal para bloques catch)

Diálogos Avanzados: Personalización y Respuestas

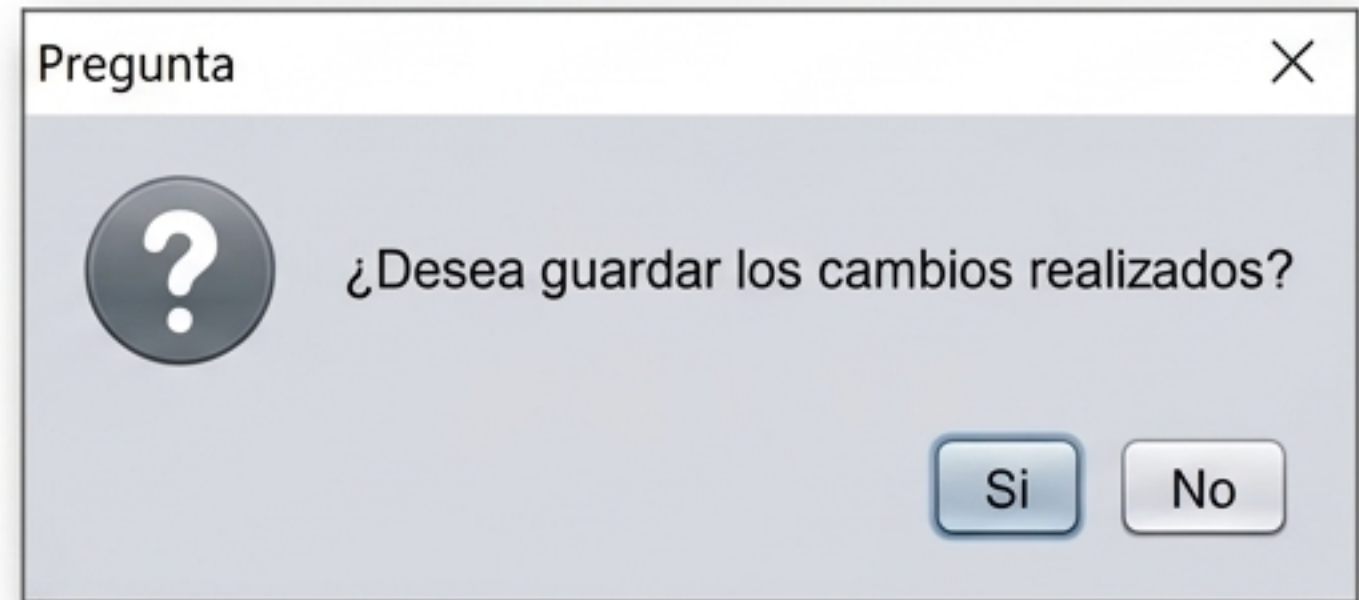
Inyección de Iconos



```
ImageIcon icono = new ImageIcon(ruta);  
JOptionPane.showMessageDialog(.., icono);
```

Carga un ImageIcon y pásalo como 5º parámetro.

Captura de Decisiones



```
Object[] opciones = {'Si', 'No'};  
int resp = JOptionPane.showOptionDialog(.., opciones);
```

Usa constante YES_NO_CANCEL_OPTION. Devuelve un entero (índice).

Interacción con el Sistema: JFileChooser

Abrir

```
// Creamos una ventana JFileChooser,
JFileChooser fileChooser = new JFileC
// Hacemos que aparezca un fichero po
fileChooser.setSelectedFile(new File(
// Cambiamos el título de la ventana
fileChooser.setDialogTitle("Seleccior
// Agregamos dos filtros al selector
FileFilter filter = new FileNameExte
fileChooser.addChoosableFileFilter(fi
FileFilter filter2 = new FileNameExte
fileChooser.addChoosableFileFilter(fi
// Mostramos la ventana de selecciona
int resultado = fileChooser.showOpenD
ejecuta
if (resultado == JFileChooser.APPROVE
```

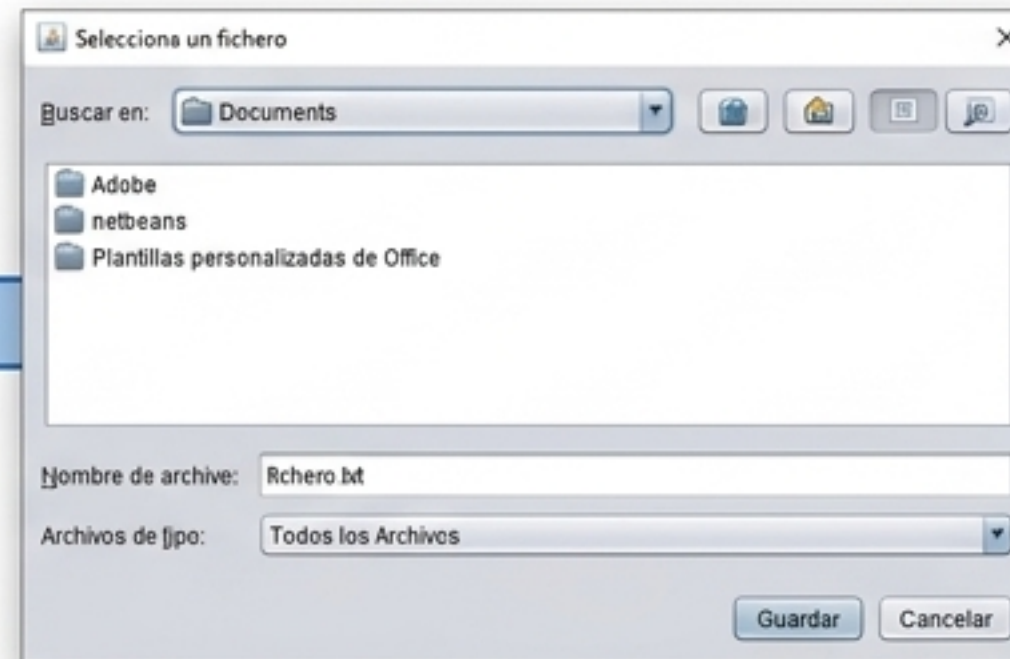
Acción: showOpenDialog.

Flujo: Validar APPROVE_OPTION y usar **getSelectedFile()**.

Guardar

```
// Creamos una ventana JFileChooser,
JFileChooser fileChooser = new JFileC
// Hacemos que aparezca un fichero po
fileChooser.setSelectedFile(new File(
// Cambiamos el título de la ventana
fileChooser.setDialogTitle("Seleccior
// Mostramos la ventana de selecciona
FileFilter filter = new FileNameExte
fileChooser.addChoosableFileFilter(fi
FileFilter filter2 = new FileNameExte
fileChooser.addChoosableFileFilter(fi
// Mostramos la ventana de selecciona
// Si se ha seleccionado un fichero,
ejecuta
if (resultado == JFileChooser.APPROVE
```

Acción: showSaveDialog.



Tip: Integra **FileNameExtensionFilter** para limitar la búsqueda a tipos específicos (ej. .txt, .java).

Arquitectura de Navegación: Menús



JMenuBar

JMenu

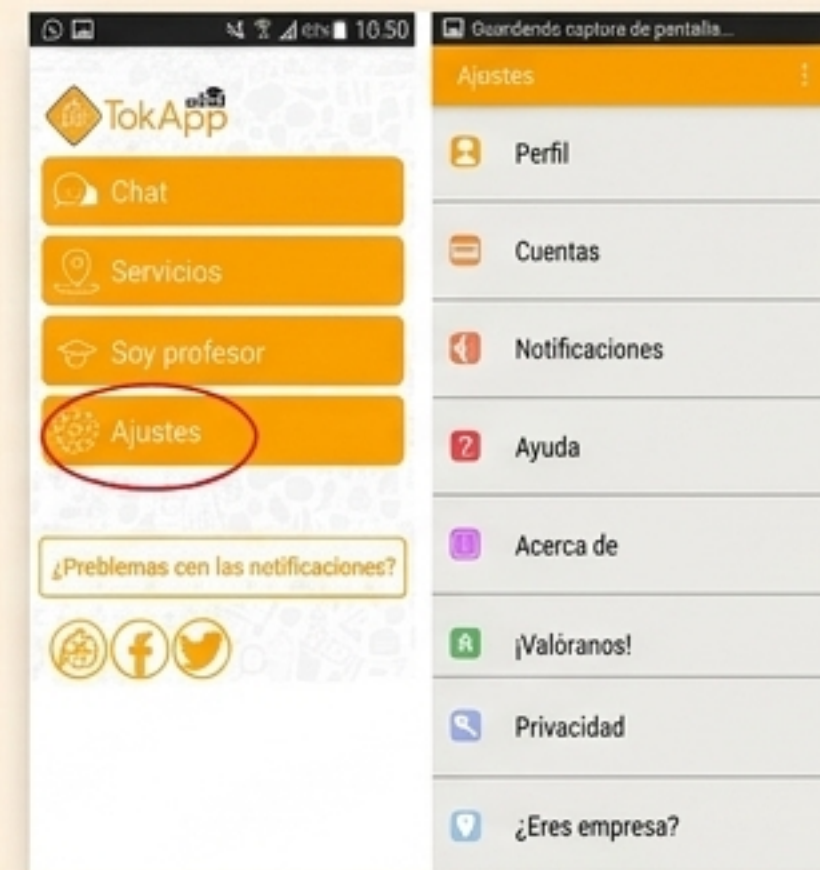
La barra raíz de la aplicación. Contiene las categorías principales (ej. File, Edit).

JMenuItem

Acciones clicables que ejecutan eventos (con `actionPerformed`).

Separator

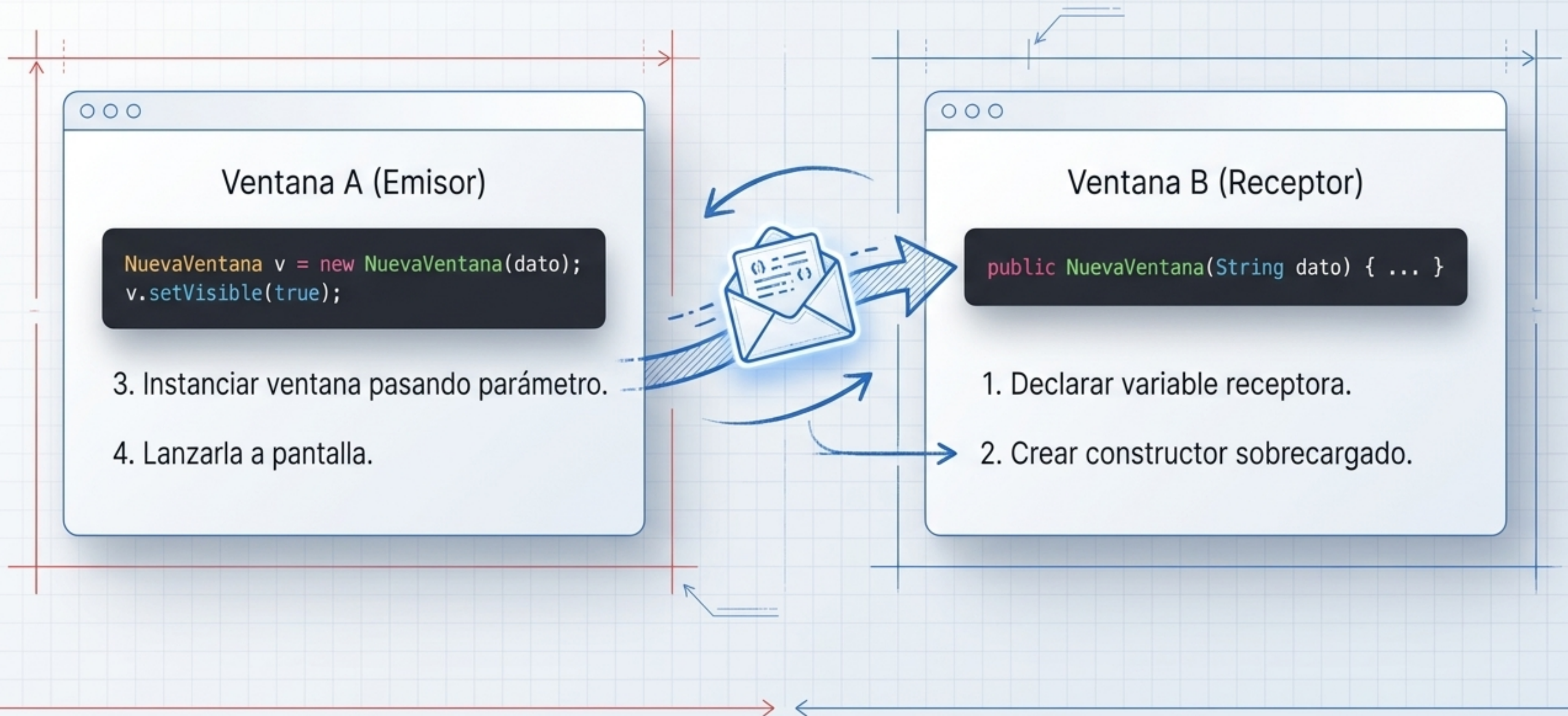
Líneas divisorias para organizar opciones.



Caso Práctico: UX

¿Cuándo NO usar menús?
Los menús tradicionales son hostiles en pantallas táctiles o apps hiper-simples. Evaluar siempre el entorno y la modernidad de la interfaz.

El Ecosistema Multi-Ventana



Mapa Mental SWING (Síntesis)

